CHAPTER 6

Sharing Functionality: Inheritance, Modules, and Mixins

One of the principles of good software design is the elimination of unnecessary duplication. We work hard to make sure that each concept in our application is expressed just once in our code. Why? Because the world changes. And when you adapt your application to each change, you want to know that you've changed exactly the code you need to change. If each real-world concept is implemented at a single point in the code, this becomes vastly easier.

We've already seen how classes help reduce duplication. All the methods in a class are automatically accessible to instances of that class. But there are other, more general types of sharing that we want to do. Maybe we're dealing with an application that ships goods. Many forms of shipping are available, but all forms share some basic functionality, perhaps weight calculation. We don't want to duplicate the code that implements this functionality across the implementation of each shipping type.

Or maybe we have a more generic capability that we want to inject into a number of different classes. For example, an online store may need the ability to calculate sales tax for carts, orders, quotes, and so on. Again, we don't want to duplicate the sales tax code in each of these places.

In this chapter, we'll look at two different but related mechanisms for this kind of sharing in Ruby. The first, *class-level inheritance*, is common in object-oriented languages. We'll then look at *mixins*, a technique that is often preferable to inheritance. We'll wind up with a discussion of when to use each.

Inheritance and Messages

In a previous chapter, we saw that when the `puts` method needs to convert an object to a string, it calls that object's `to_s` method. But we've also written our own classes that don't explicitly implement `to_s`. Despite this, instances of these classes respond successfully when we call `to_s` on them. How this works has to do with inheritance and how Ruby uses inheritance to determine what method to run when you send a message to an object.

Inheritance allows you to create a class that is a refinement or specialization of another class. This specialized class is called a *subclass* of the original, and the original is a *superclass* of the subclass. People also refer to this relationship as *child* and *parent* classes.